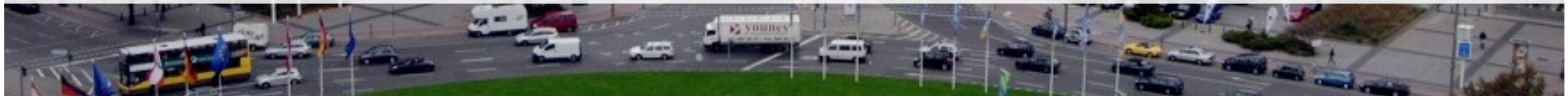


MODUL KÜNSTLICHE INTELLIGENZ: GRUNDLAGEN UND ANWENDUNGEN

MASCHINELLES LERNEN MIT NEURONALEN NETZEN, TEIL 1

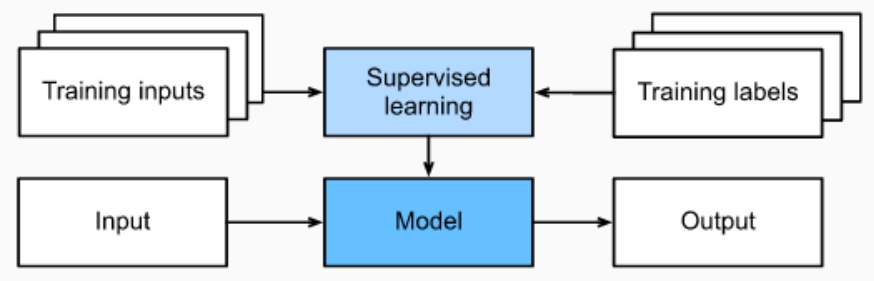
Einleitung | Perzeptron| Perzeptron-Lernregel | Backpropagation | Trainingsprozess | Overfitting | Ausblick

Dr.-Ing. Stefan Fricke | Fachgebiet *Service-centric Networking* | TU Berlin

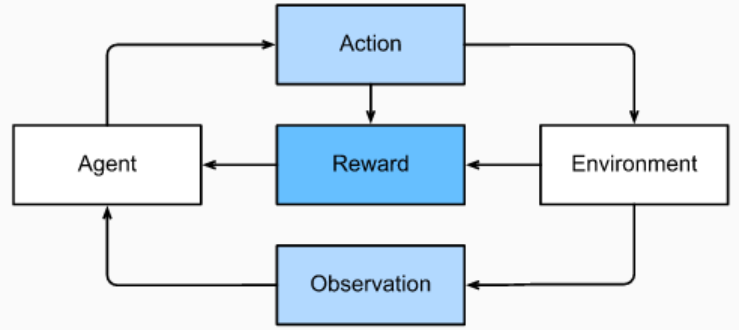


Maschinelle Lernverfahren im Vergleich

Supervised Learning =
Informationsextraktion aus Daten



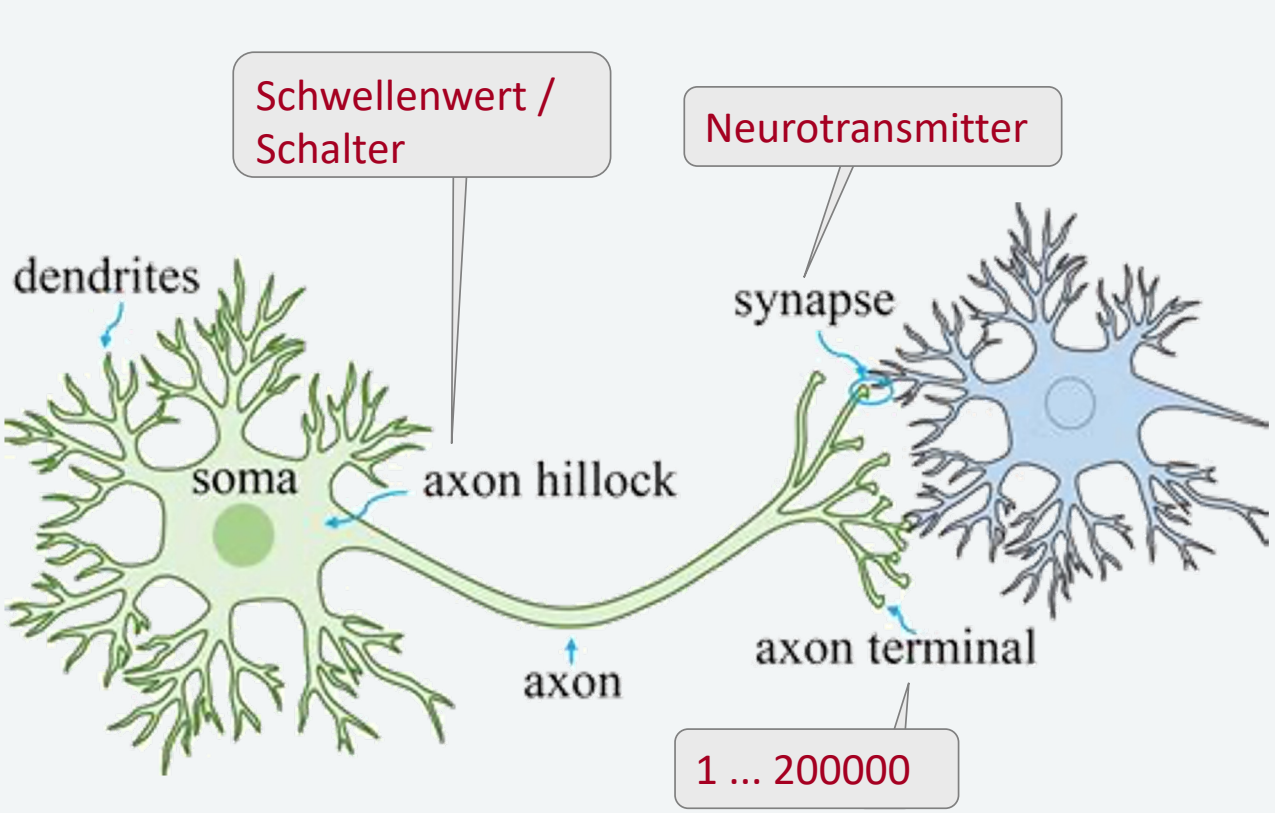
Reinforcement Learning =
Interaktion mit Umgebung

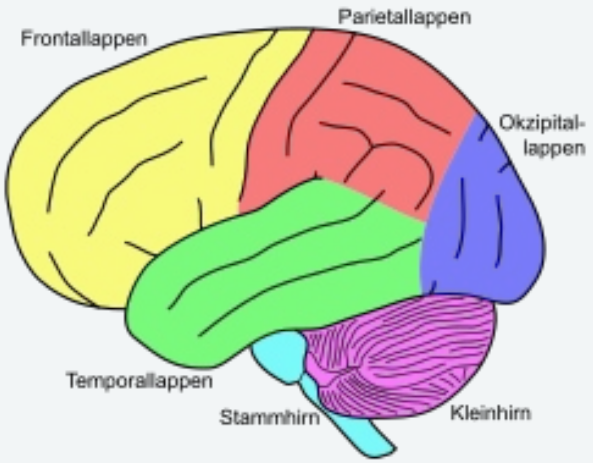


Außerdem: Unsupervised Learning, Caching, genetische Algorithmen, Induktion

Lernen	vs.	Programmieren
Training	vs.	Einsatz
Klassifikation	vs.	Regression

Natürliche Neuronen verarbeiten elektro-chemische Signale



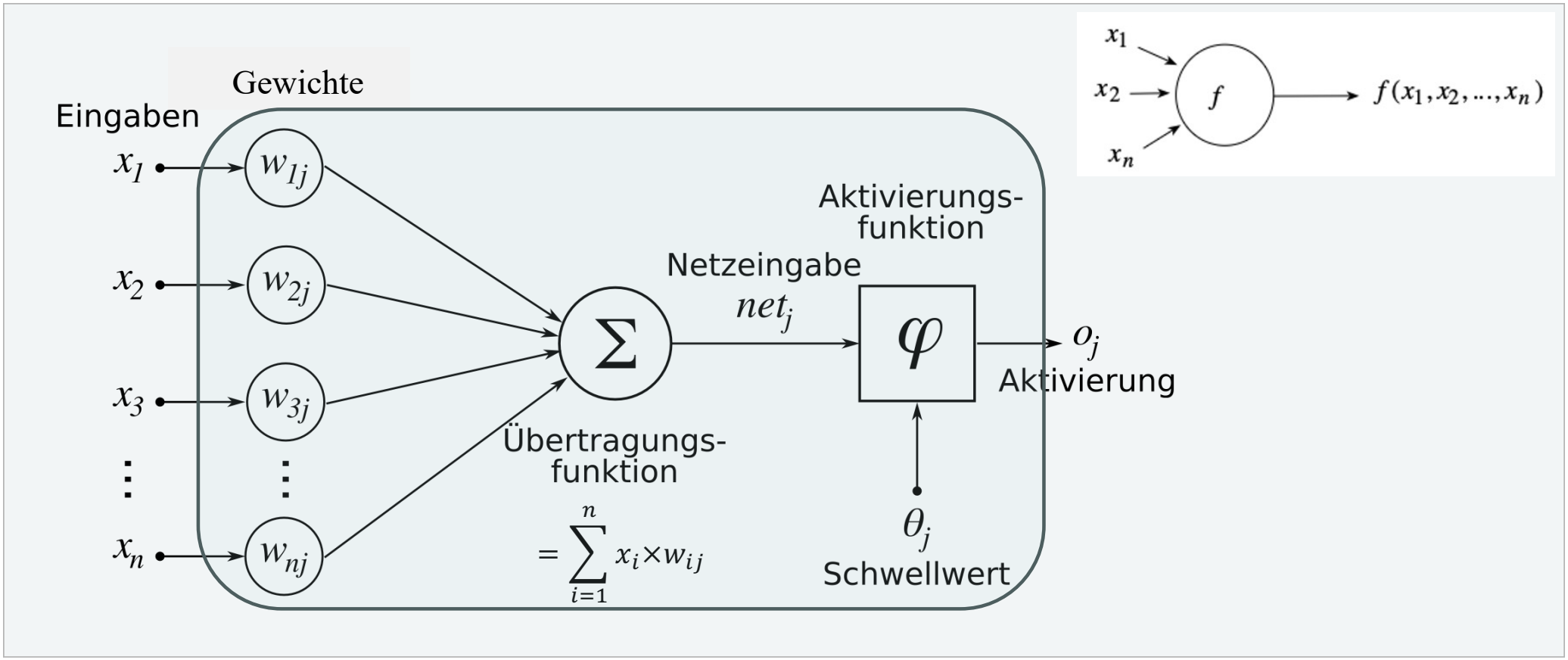


Menschliches Gehirn:

- 10^{11} Neuronen
- 10^{14} Synapsen
- 20 Watt Leistung
- 10^{13} FLOPS (parallel)

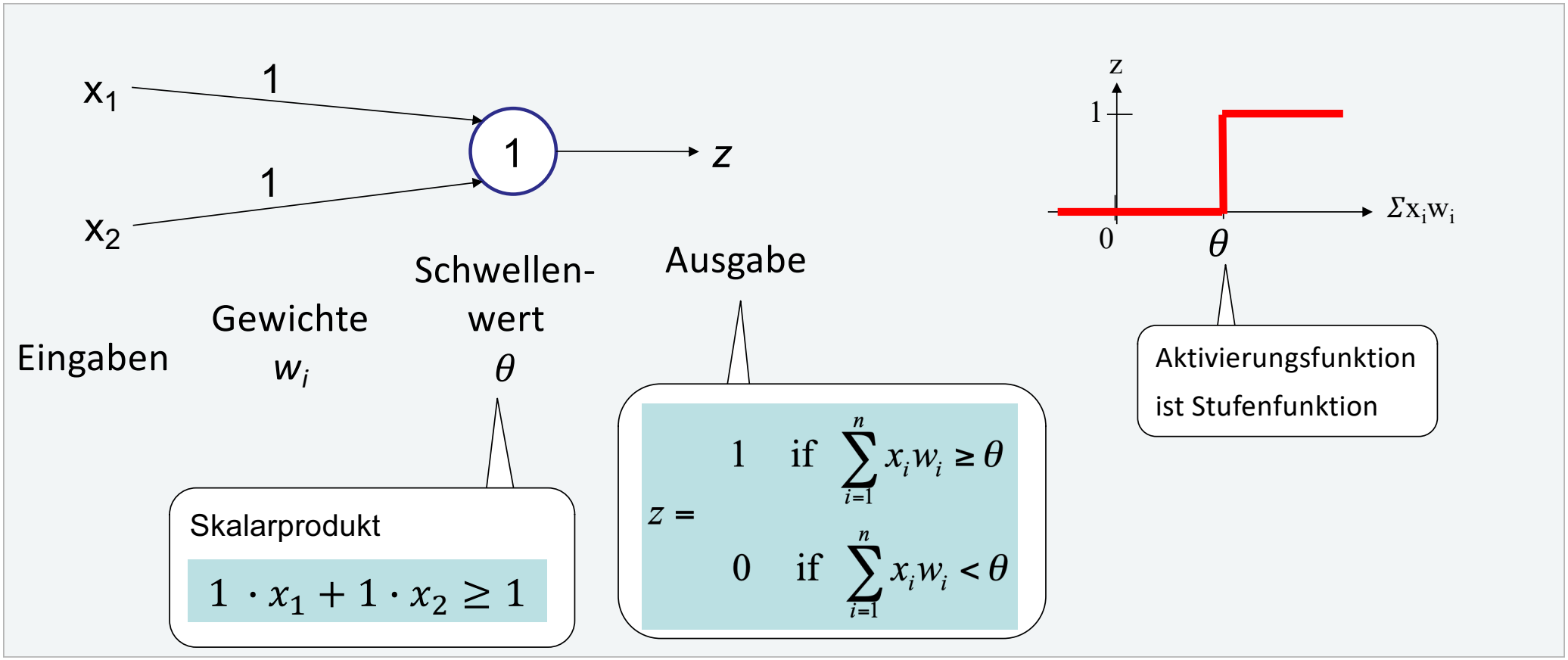
Von NEUROtiker - Eigenes Werk, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=2650117

Ein künstliches Neuron j berechnet für einen Eingabevektor x einen Ausgabewert o



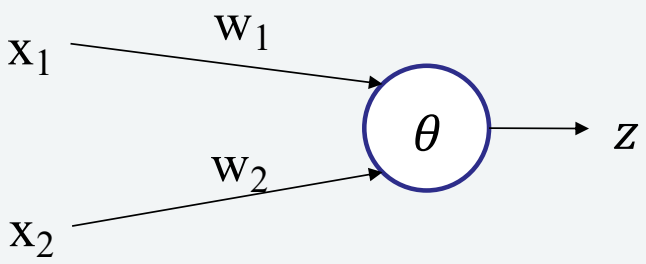
Erstellt von Chrislb, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=224561

Ein Perzeptron verwendet *gewichtete* Eingaben

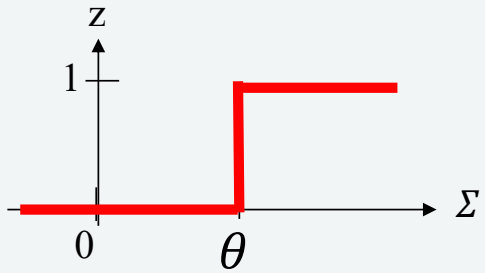


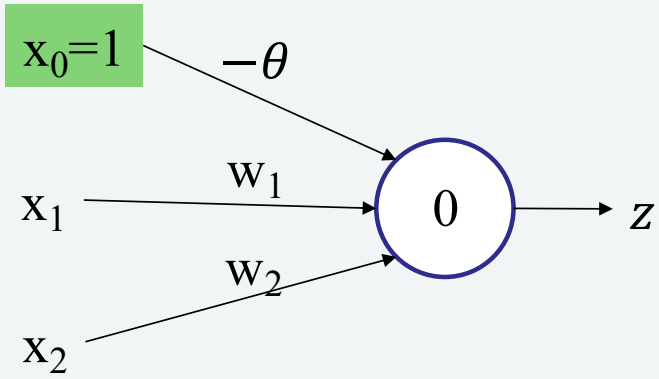
F. Rosenblatt: The perceptron – a probabilistic model for information storage and organization in the brain. Psychological Review 65, 1958.

Bias-Neuron x_0 ersetzt den Schwellenwert

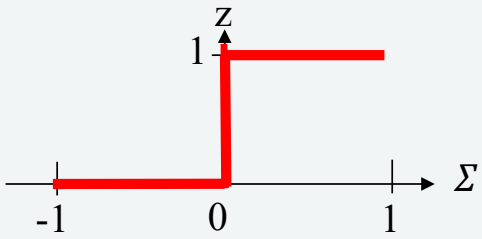


$$z = \begin{cases} 1 \ if \ \sum_{i=1}^{n} x_i w_i \geq \theta \\ 0 \ else \end{cases}$$

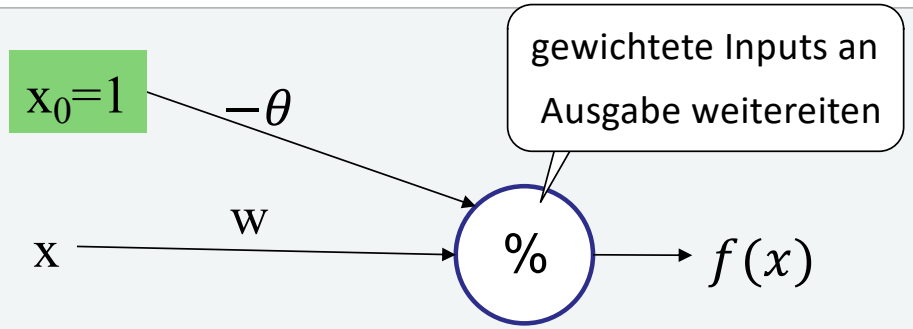




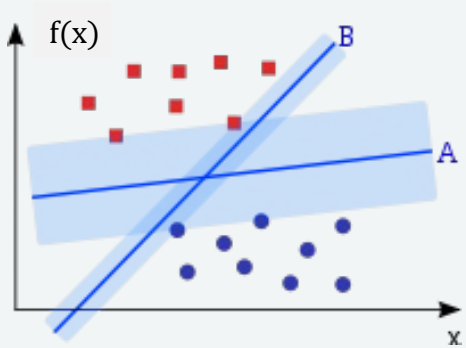
$$z = \begin{cases} 1 \ if \ \sum_{i=0}^{n} x_i w_i \geq 0 \\ 0 \ else \end{cases}$$



Die Summe der gewichteten Eingaben ergibt eine lineare Funktion



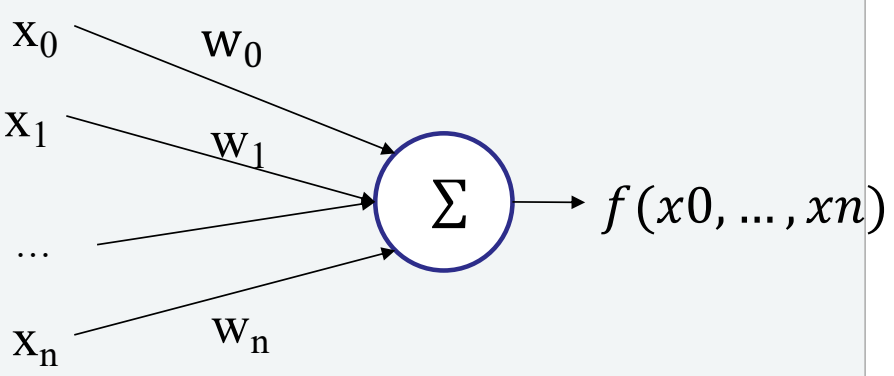
$f(x) = wx - \theta$ ist lineare Funktion



B: $x - 1$

A: $0.2\,x + 4$

Allgemeiner Fall:



Sei $X=(x_0,\ldots,x_n)$ und $W=(w_0,\ldots,w_n)$.

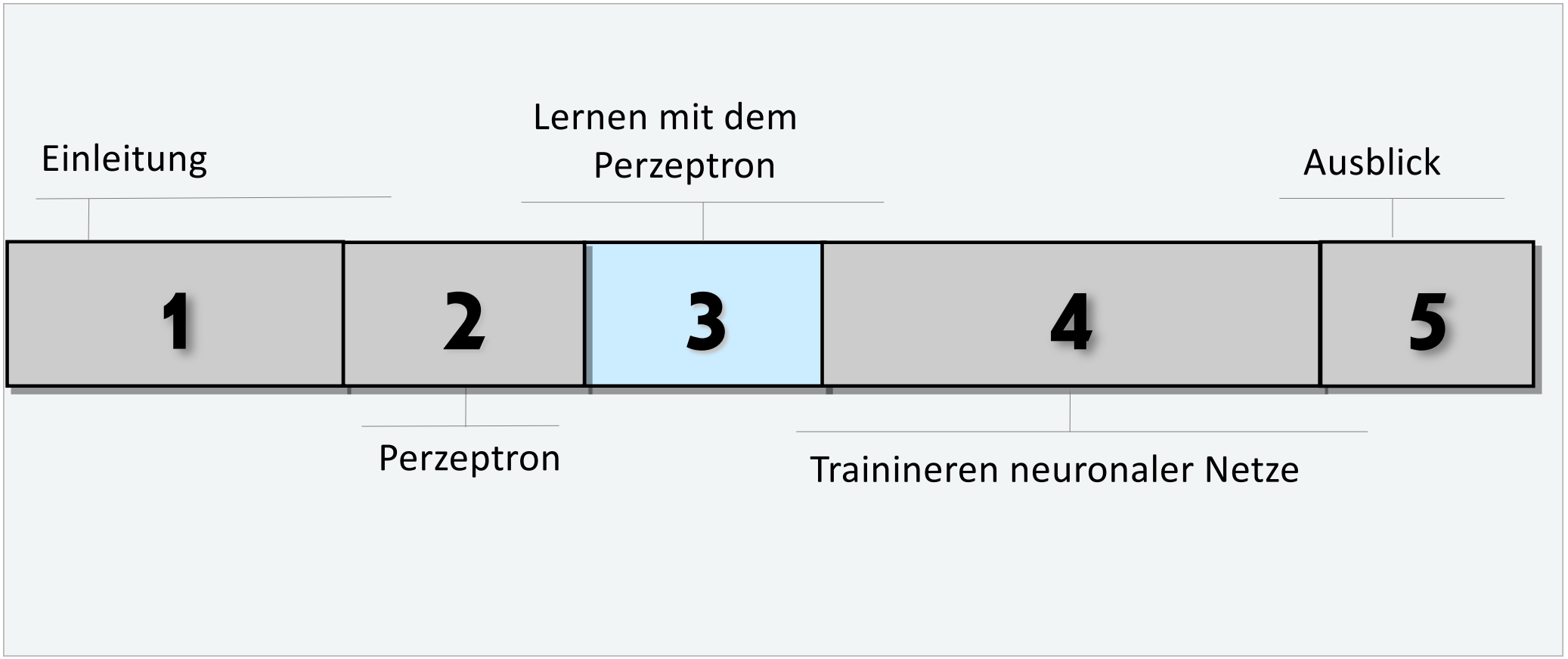
Dann ist $f(X) = W \cdot X$

ebenfalls eine lineare Funktion

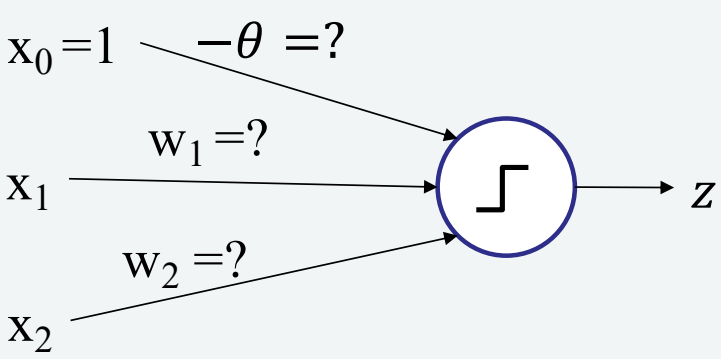
Output bei Verwendung der Stufenfunktion ist lineare Separierung

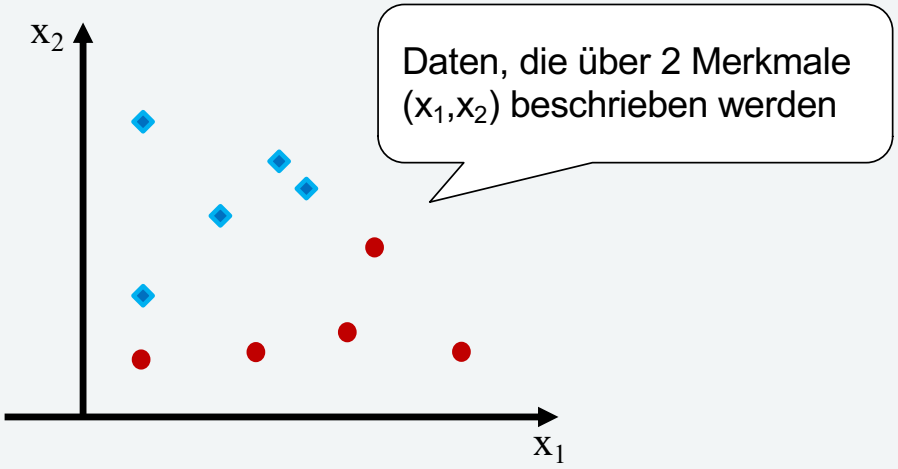
Wichtigste Anwendung: Klassifikation

Agenda maschinelles Lernen, Teil 1



Anwendungsfall binäre Klassifikation linear korrelierter Daten





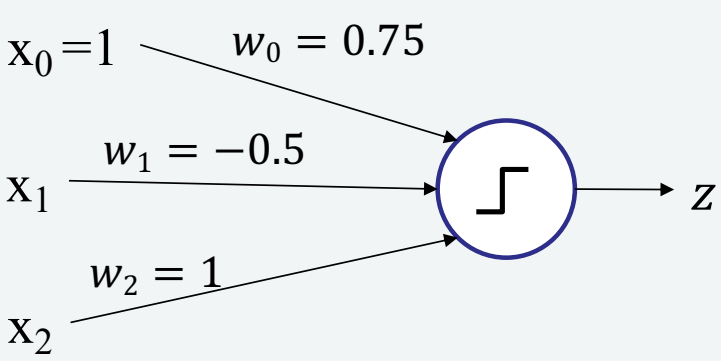
x_i: Merkmale

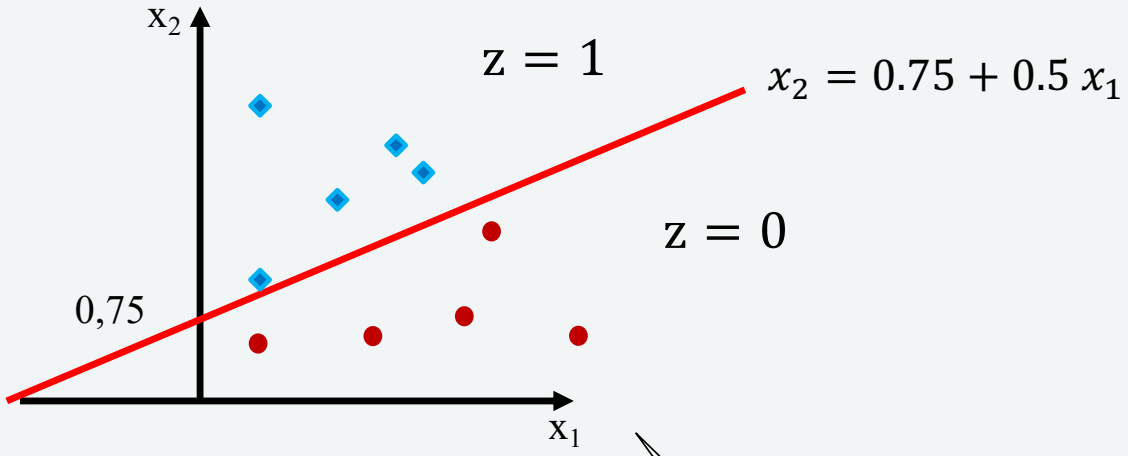
z: Klassenzugehörigkeit (ja/nein)

● $z = 0$ ◆ $z = 1$

Aufgabe: Klassifikator anhand von Beispielen trainieren

Ein Perzeptron kann lineare Separierung darstellen





x_i: Merkmale

z: Klassenzugehörigkeit (ja/nein)

$z = 0$ $z = 1$

Aufgabe: Klassifikator anhand von Beispielen trainieren

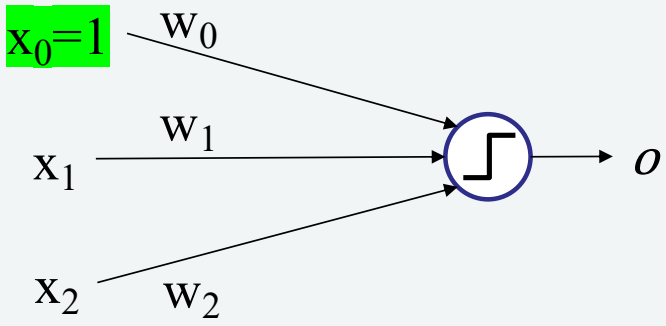
(x_0, …, x_2) ergibt lineare Funktion

(x_0, …, x_3) ergibt Trennebene

(x_0, …, x_4) ergibt Hyperebene

usw.

Lernen ist Anpassung der Gewichte anhand gelabelter Trainingsdaten



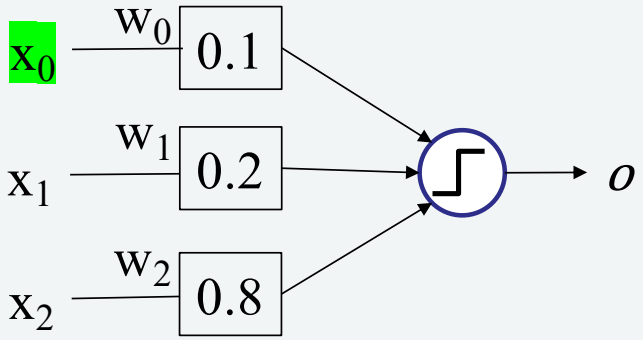
$$v_1 = ([1,1,0], 1)$$
$$v_2 = ([1,0,0], 0)$$
$$v_3 = ([1,0,1], 1)$$
$$v_4 = ([1,1,1], 1)$$
$$\ldots$$

Trainingsprozess

1. Zufällige Initialisierung der Gewichte
2. Verarbeitung der Trainingsdaten
   - Anlegen eines Trainingsvektors $v_i = (X_i, t_i)$
   - Berechnung von o_i
   - Anpassung der Gewichte ...

Input Label

Perzeptron-Lernregel: Gewichtsanpassung unter Berücksichtigung des Fehlers und der Lernrate α



$$o = \begin{cases} 1 \; if \sum_{i=0}^{n} x_i \, w_i \geq 0 \\ 0 \; else \end{cases}$$

$v_1 = ([1,1,0], 1)\,;\; o_1 = 1$

$v_2 = ([1,0,0], 0)\,;\; o_1 = 1$

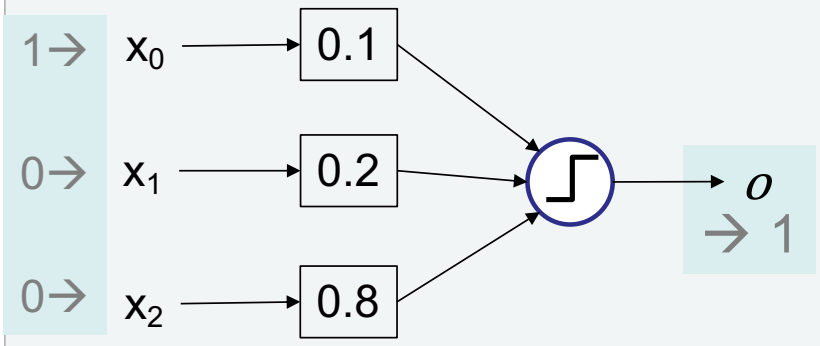
Trainingsprozess

1. Verarbeitung der Trainingsdaten
   - Anlegen eines Trainingsvektors $v_i = (X_i, t_i)$
   - Berechnung von o_i
   - Anpassung der Gewichte wenn $o_i \neq t_i$

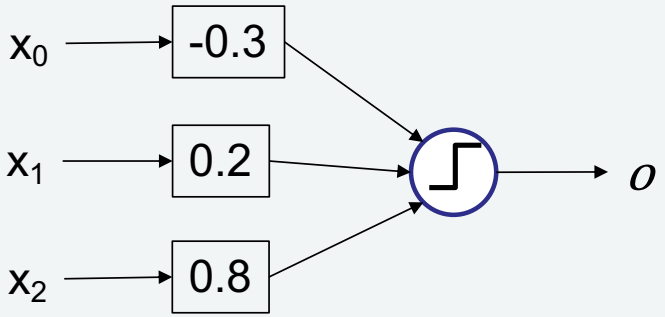
     $w_{ij}^{t+1} = w_{ij}^{t} + \delta_t \cdot \alpha \cdot x_i$ mit $\delta_t = t_i - o_i$

Anpassung der Gewichte für Trainingssatz $v_2 = ([1,0,0], 0)$

Zeitpunkt $t = 2$



Zeitpunkt $t = 3$



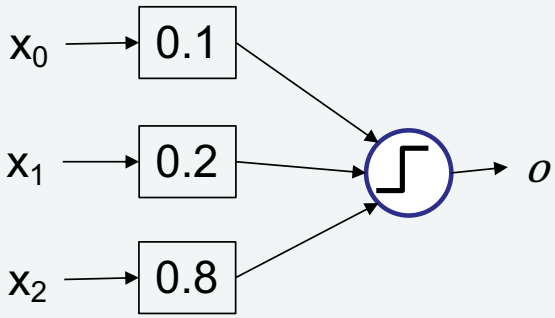
$w_{ij}^3 = w_{ij}^2 + \delta_2 \cdot \alpha \cdot x_i$ mit $\delta_2 = t_2 - o_2 = -1$ gegeben $\alpha = 0.4$

$w_0^3 = 0.1 - 1 \cdot 0.4 \cdot 1 = -0.3$ w_1^3 und w_2^3 unverändert, weil $x_1 = x_2 = 0$

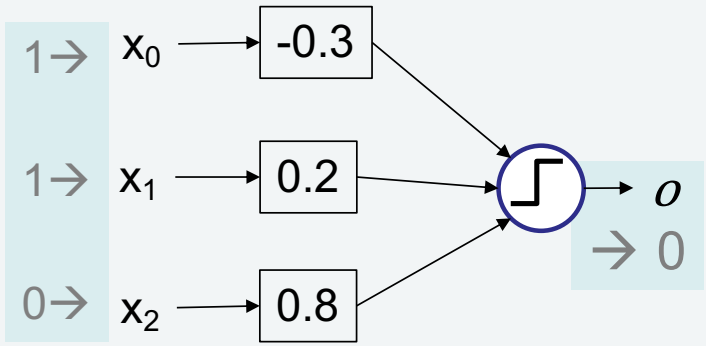
Fortsetzung des Trainings mit $v_5 = ([1,1,0], 1)$

erneut Trainingsdatensatz 1 anlegen...

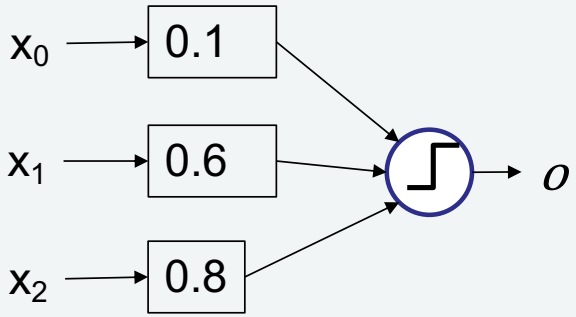
Zeitpunkt $t = 2$



Zeitpunkt $t = 3, \ldots, t = 5$



Zeitpunkt t =6

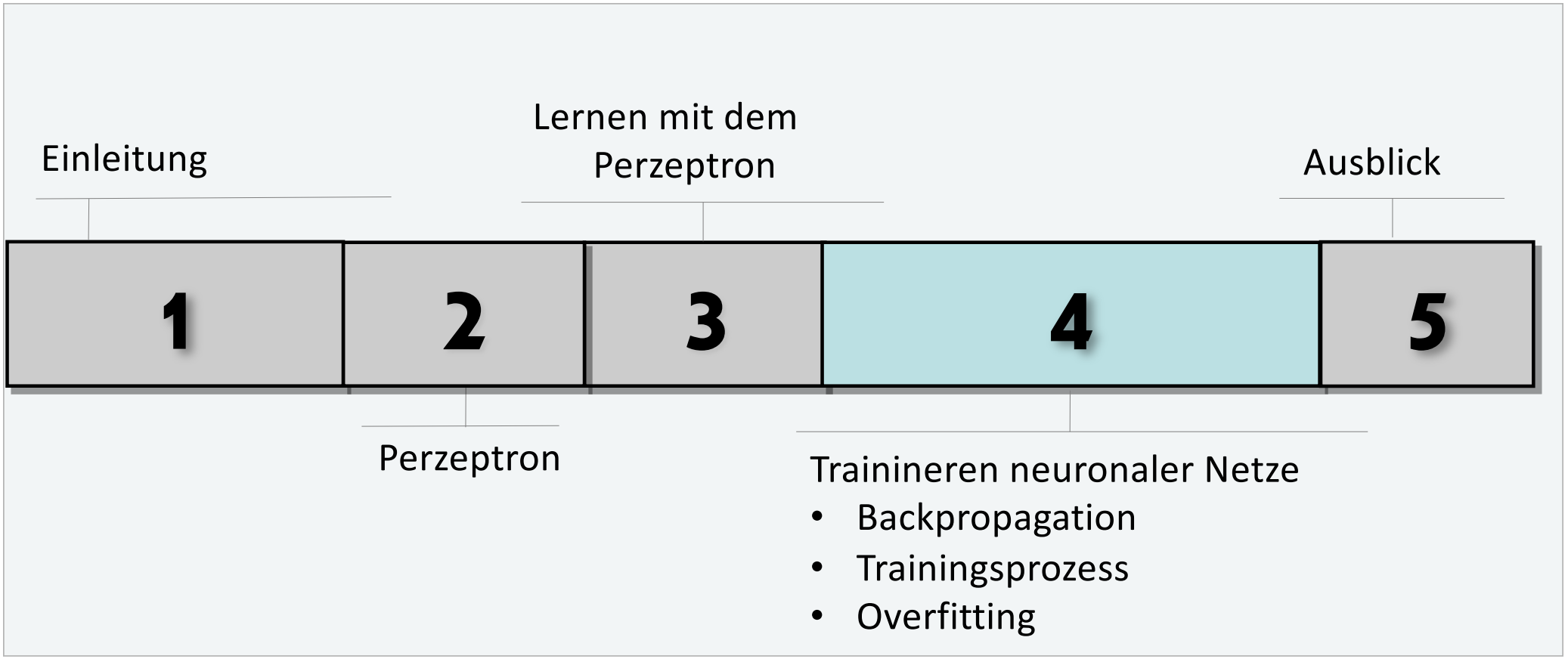


$w_{ij}^{t+1} = w_{ij}^{t} + \delta_t \cdot \alpha \cdot x_i$ mit $\delta_t = t_i - o_i$ gegeben $\alpha = 0.4$

$v_5 = ([1,1,0], 1); \quad \delta_5 = 1$

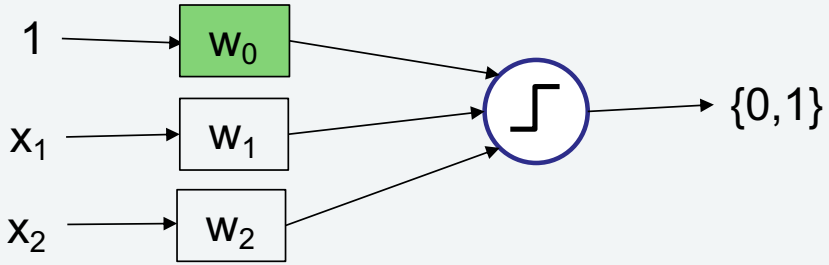
$w_0^6 = -0.3 + 0.4 = 0.1 \quad w_1^6 = 0.2 + 0.4 = 0.6$ w_2^6 unverändert

Agenda maschinelles Lernen, Teil 1

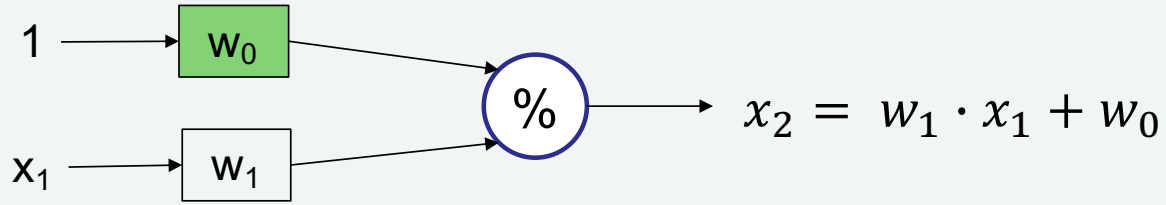


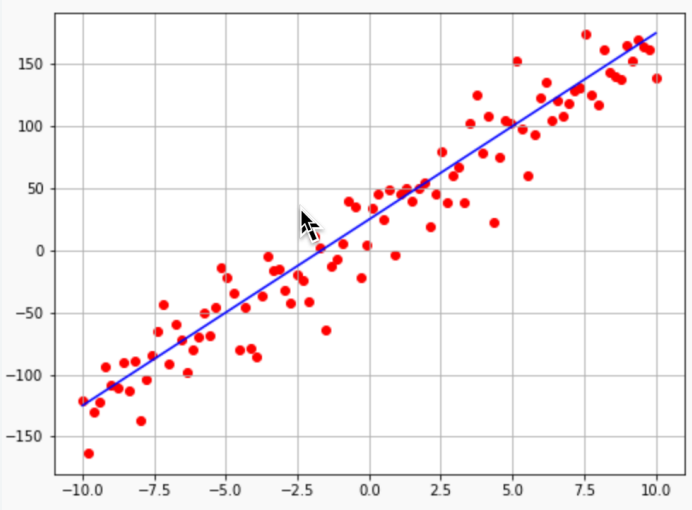
Anwendungsfall Regression: Neuron erlernt lineare Funktion

mit Stufenaktivierungsfunktion: binäre Klassifikation



mit Durchreichen des Skalarproduktes: lineare Funktion

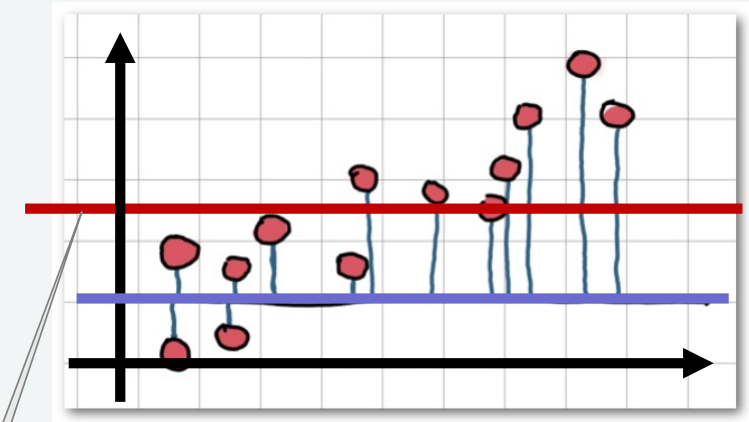




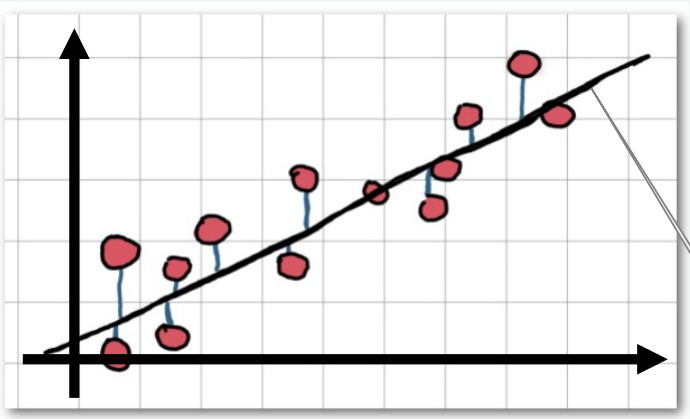
Bildquelle rechts: neuromant.de/2018/12/16/Tutorial_Das-Perzeptron-Teil-3/

Gute Modelle minimieren den Fehler zu *allen* Datenpunkten

Erlernen einer linearen Funktion zur Approximierung einer Datenwolke



schlechtes Modell: $0 \cdot x + w_0$



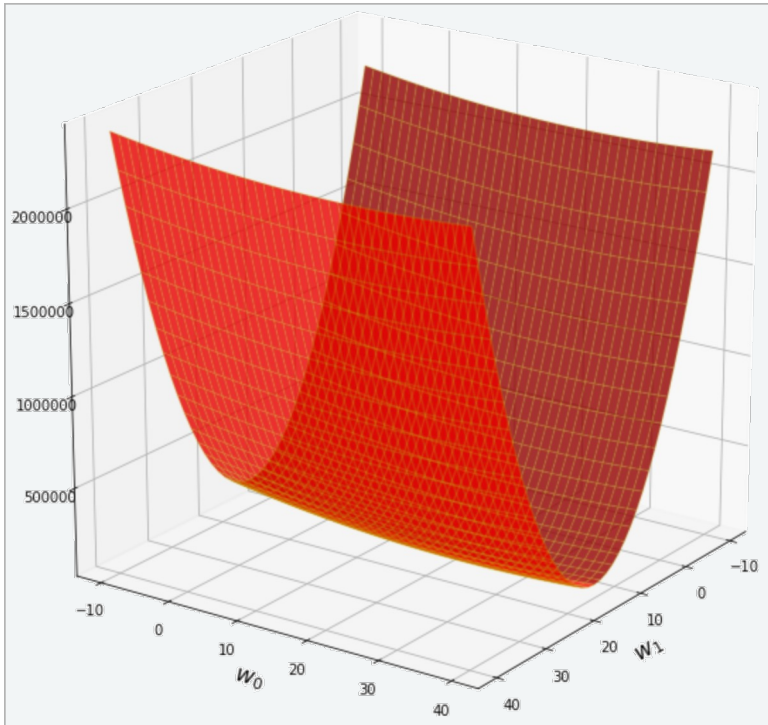
gutes Modell: $0.5 \cdot x + 0.3$

Minimierung des durchschnittlichen Fehlers

Minimierung der quadratischen Fehlerabweichung

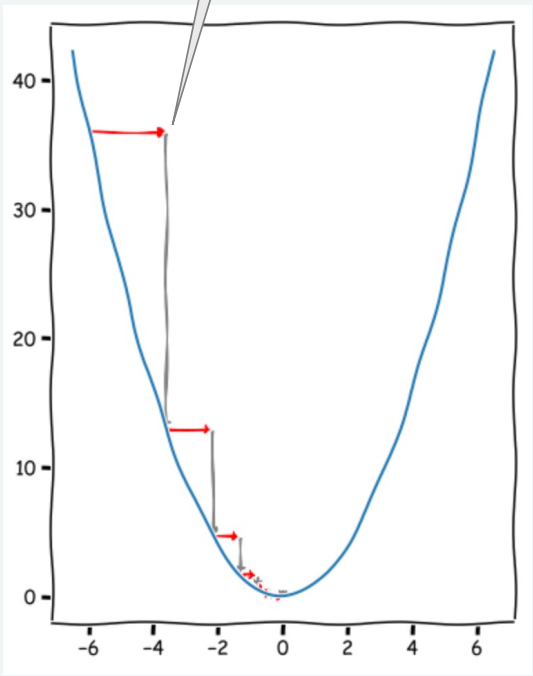
Bilderquelle: neuromant.de/2018/12/16/Tutorial_Das-Perzeptron-Teil-3/

Gradientenabstieg ist allgemeines Näherungsverfahren für neuronale Netze

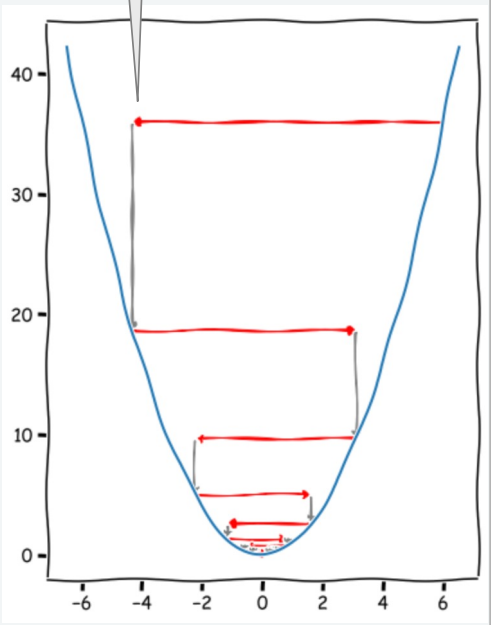


Quadratische Verlustfunktion
für Modell $w_1 \cdot x + w_0$

Folge dem steilsten Weg nach unten



Gradientenabstieg mit
kleiner Lernrate $\alpha = 0.2$



Gradientenabstieg mit
hoher Lernrate $\alpha = 0.8$

Quelle: neuromant.de/2018/12/16/Tutorial_Das-Perzeptron-Teil-3/

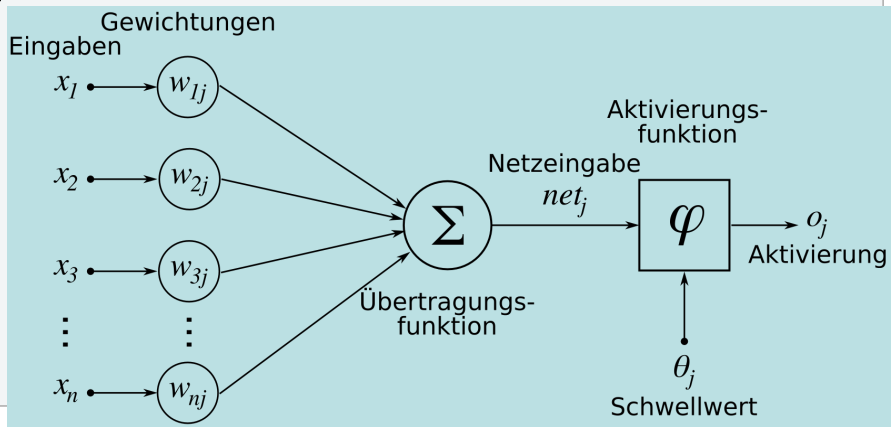
Die Perzeptron-Lernregel (*Delta-Regel*) realisiert den Gradientenabstieg

Delta-Regel mit Stufenaktivierungsfunktion ⎍ :

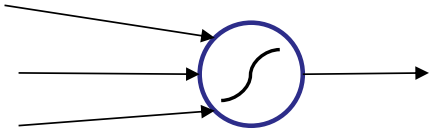
$$w_{ij}^{t+1} = w_{ij}^{t} + \delta_t \cdot \alpha \cdot x_i \qquad \text{mit } \delta_t = t_i - o_i \text{ und Lernrate } \alpha$$

Delta-Regel mit beliebiger Aktivierungsfunktion $\boldsymbol{\varphi}$:

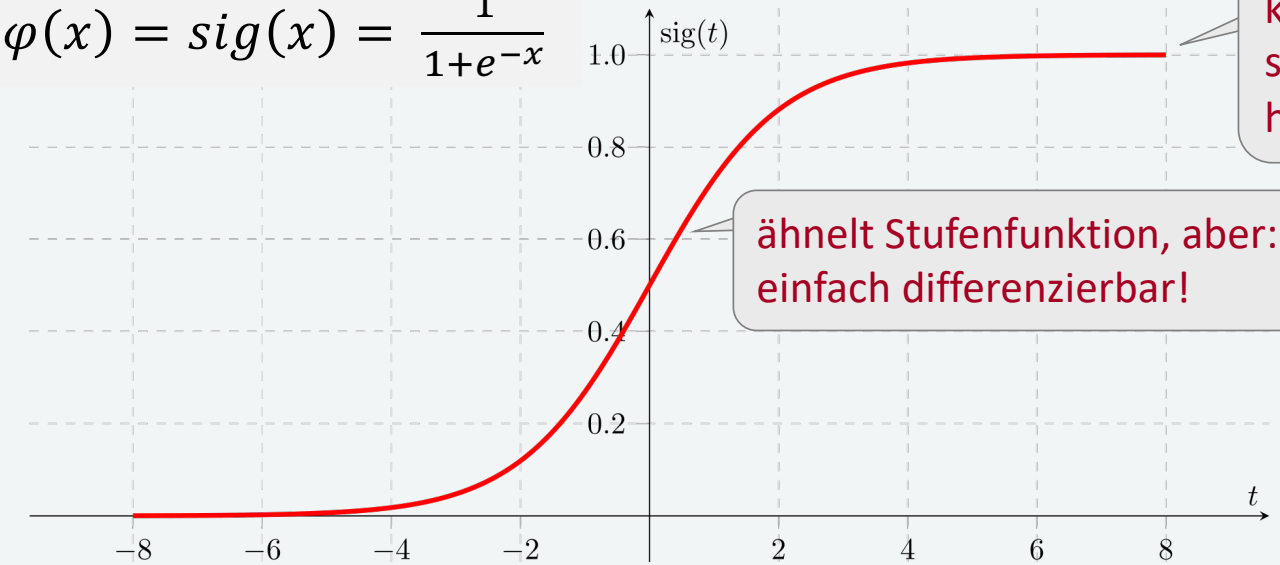
$$w_{ij}^{t+1} = w_{ij}^{t} + \delta_t \cdot \alpha \cdot x_i \cdot \frac{\partial \varphi(net_j)}{\partial net_j}$$



Sigmoid ist eine häufig verwendete Aktivierungsfunktion



$$\varphi(x) = sig(x) = \frac{1}{1+e^{-x}}$$



kleine Änderungen in den Gewichten sollen kleine Änderungen im Output hervorrufen

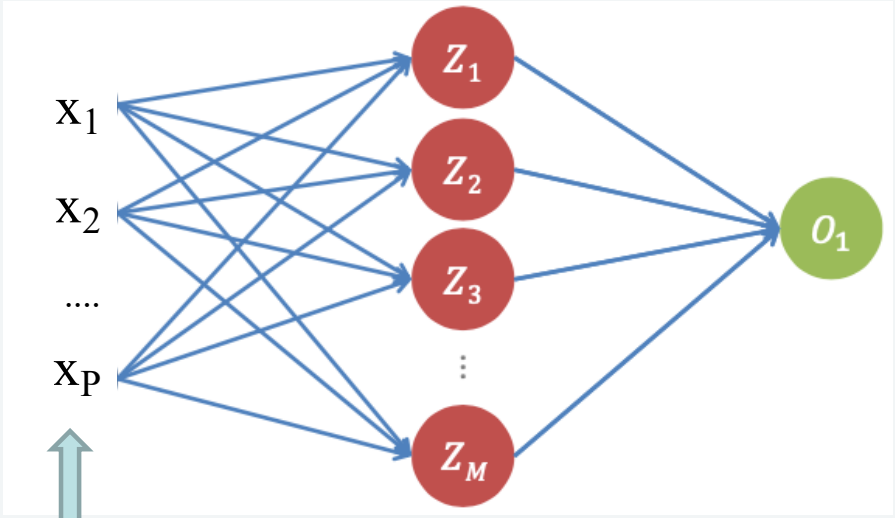
ähnelt Stufenfunktion, aber: einfach differenzierbar!

$$sig'(x) = sig(x) \cdot \big(1 - sig(x)\big)$$

Von MartinThoma - Eigenes Werk, CC0, https://commons.wikimedia.org/w/index.php?curid=32970804

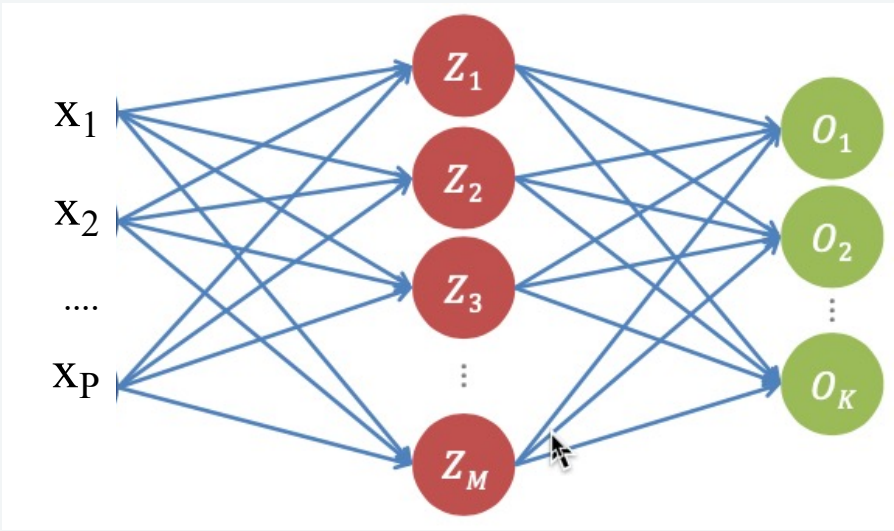
Die typische Topologie eines einfachen neuronalen Netzes beinhaltet *eine* verdeckte Schicht

Regression: Wertvorhersage



Eingabeschicht
(Feature-Vektor, Merkmale, Inputparameter)

Klassifikation: Klassenzugehörigkeit



Eingabe-(schicht)

hidden layer

Ausgabe-schicht

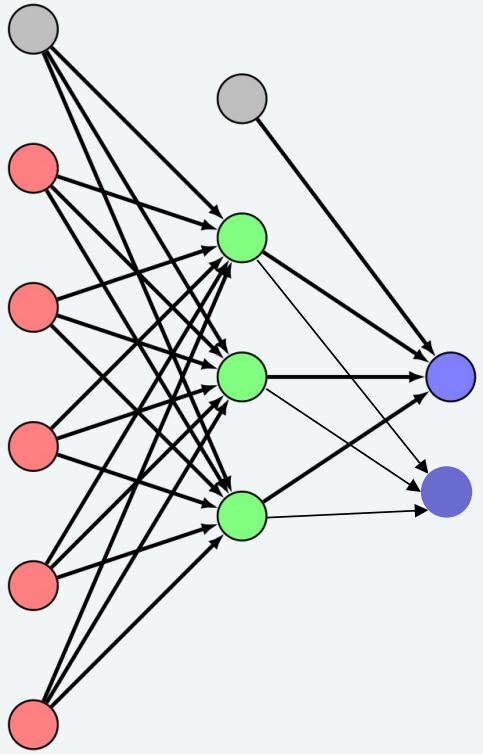
Backpropagation definiert Lernregel für mehrschichtige neuronale Feedworwardnetze

Erweiterung der Delta-Regel auf mehrere Schichten notwendig:

$$\Delta w_{ij} = \alpha \cdot x_i \cdot \delta_j$$

mit $\delta_j = \varphi'(net_j) \cdot (t_j - o_j)$ für Ausgabeneuron j

$\delta_j = \varphi'(net_j) \cdot \sum_k \delta_k \cdot w_{jk}$ für inneres Neuron j



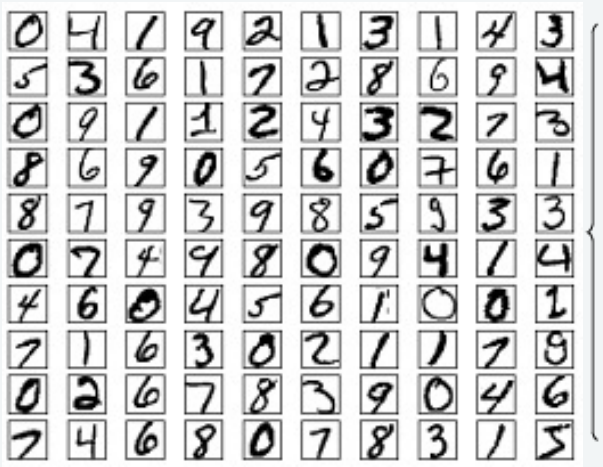
Martin Thoma, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=27872538

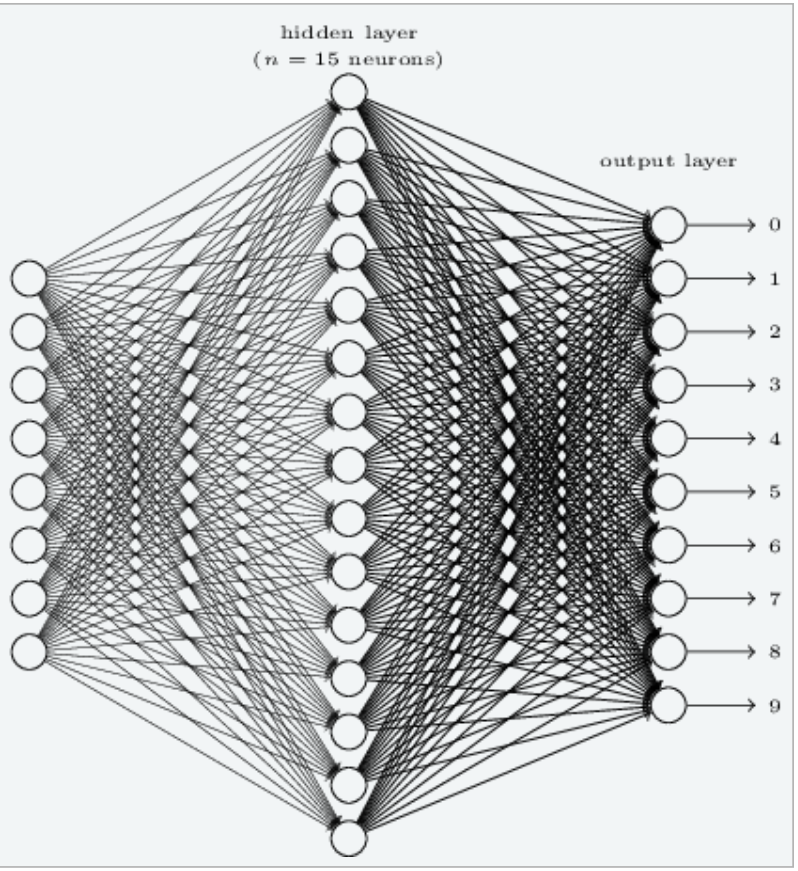
Anwendungsfall: Handschrifterkennung

Input: 28 x 28 Pixel

1 verdeckte Schicht mit 15 Neuronen, vollständig vernetzt

Output: 10 Neuronen



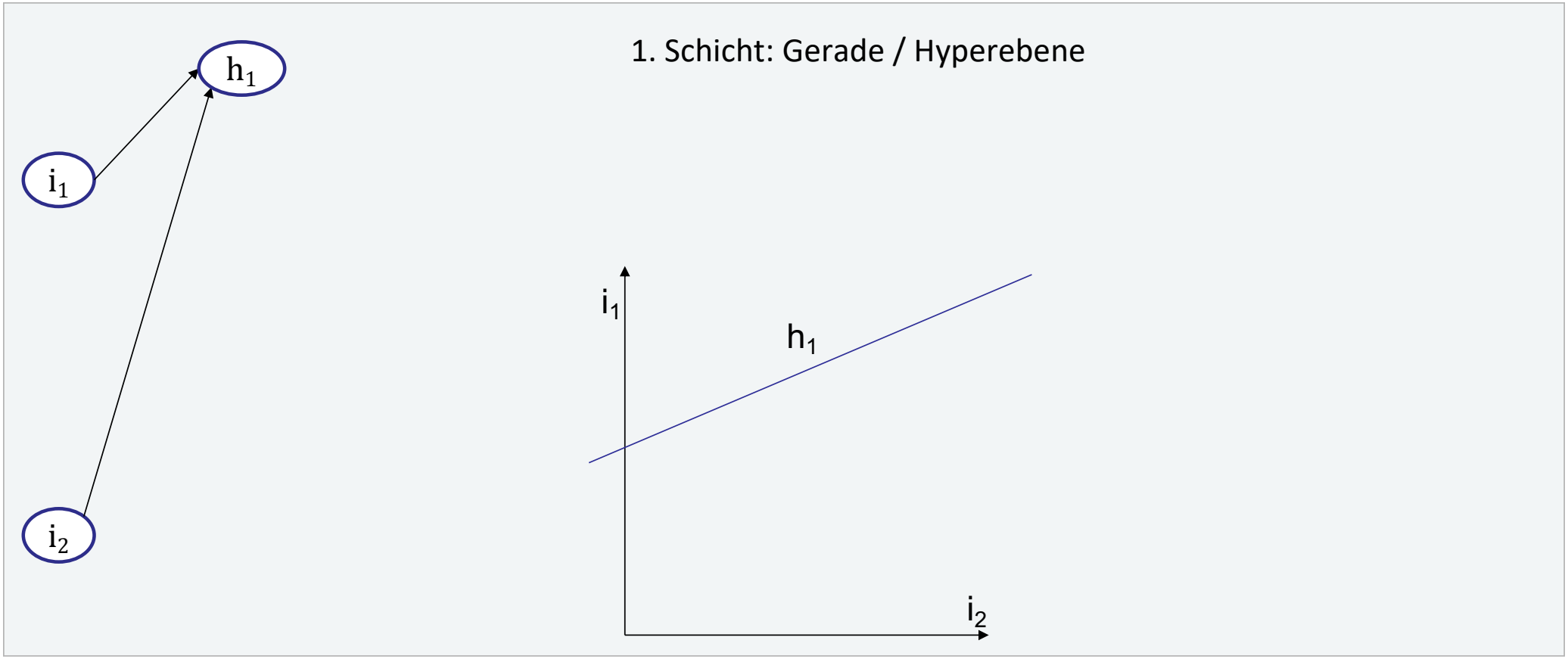


viele Designentscheidungen:
→ Hyperparametertuning

MNIST ist gelöst.

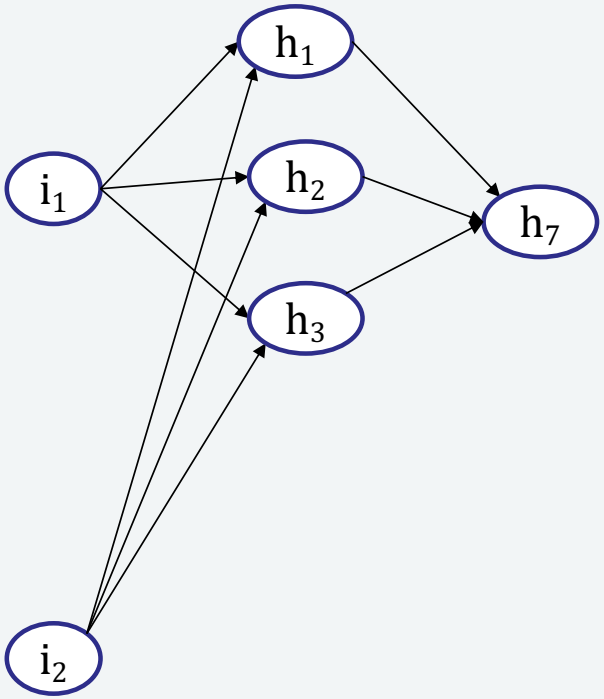
MNIST: 60,000 Beispiele
28×28 pixel, Graustufen

In der Theorie genügen 3 Schichten



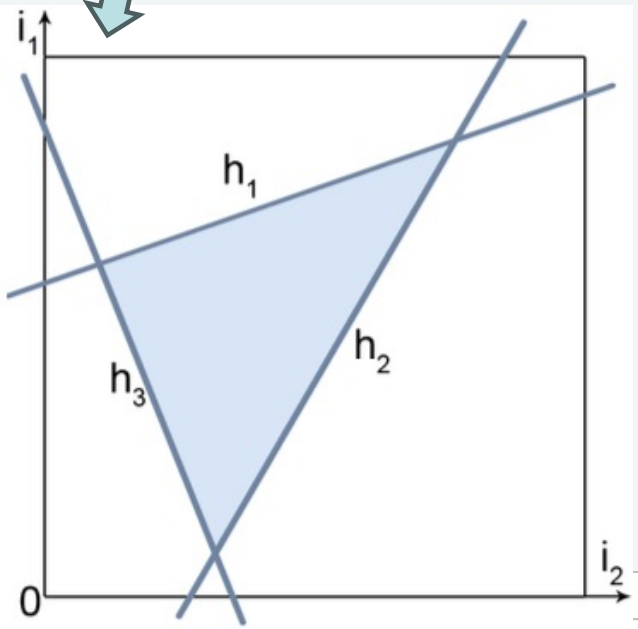
D. Kriesel (2007), Abbildung 5.10

In der Theorie genügen 3 Schichten



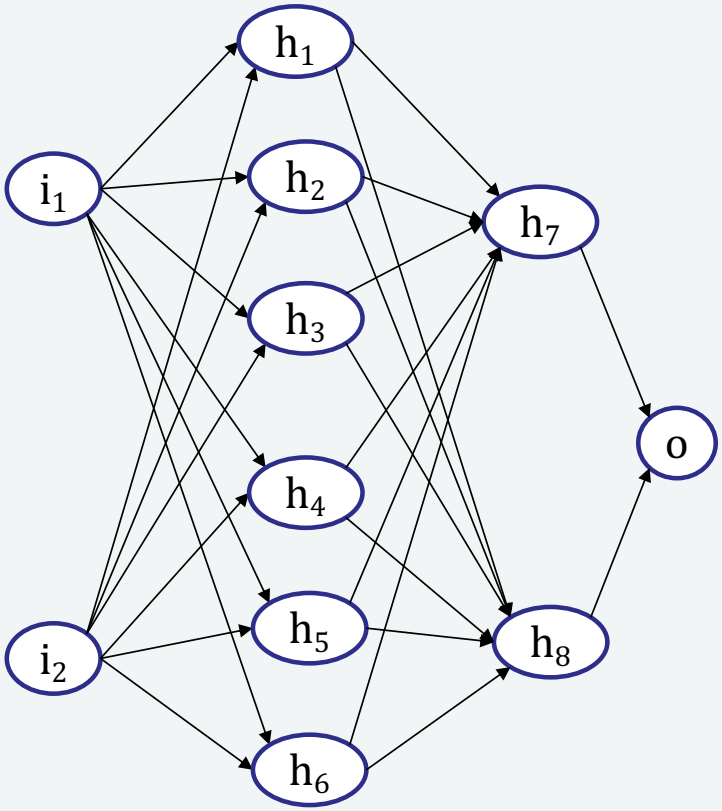
1. Schicht: Gerade / Hyperebene

2. Schicht: konvexes Polygon



D. Kriesel (2007), Abbildung 5.10

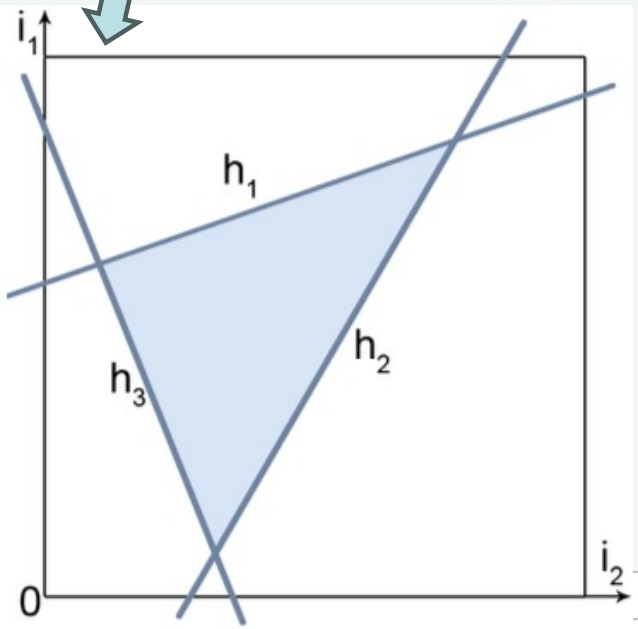
In der Theorie genügen 3 Schichten

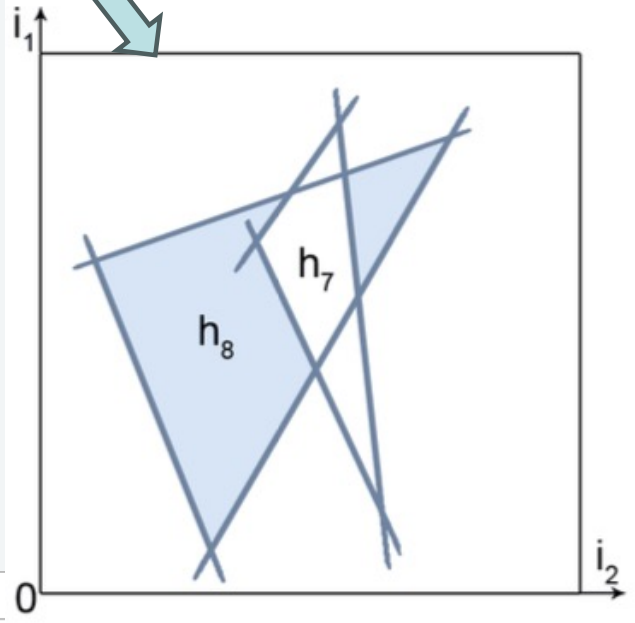


1. Schicht: Gerade / Hyperebene

2. Schicht: konvexes Polygon

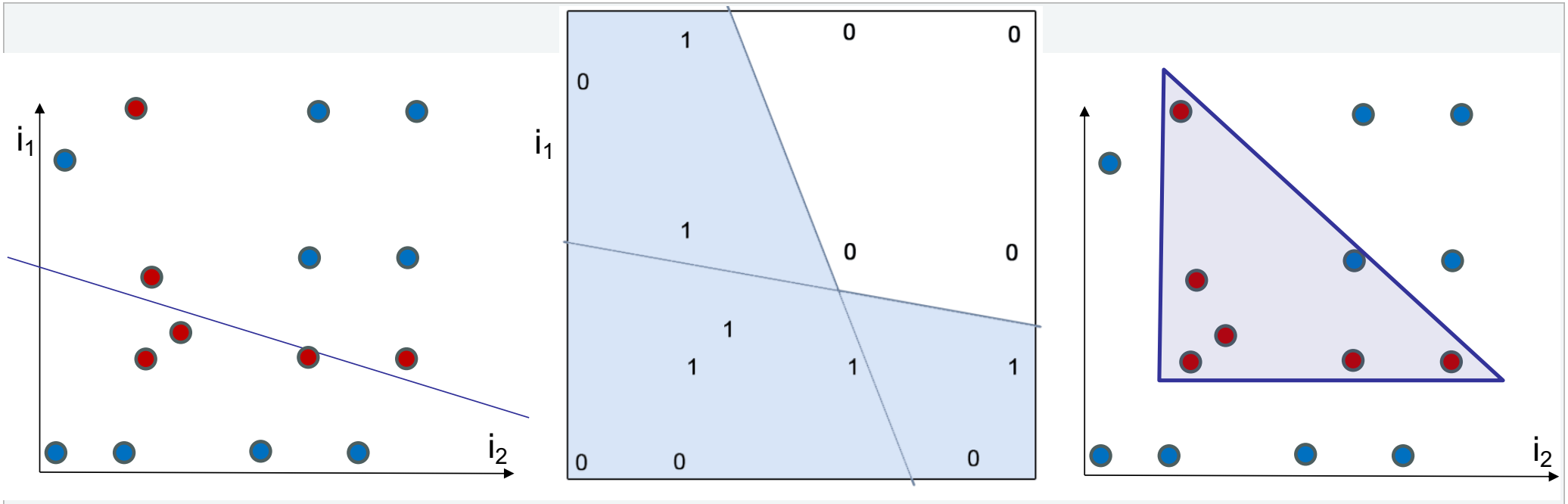
3. Schicht: beliebige Menge





D. Kriesel (2007), Abbildung 5.10

Underfitting entsteht bei zu einfachen Netztopologien oder zu wenig Training



unterkomplex

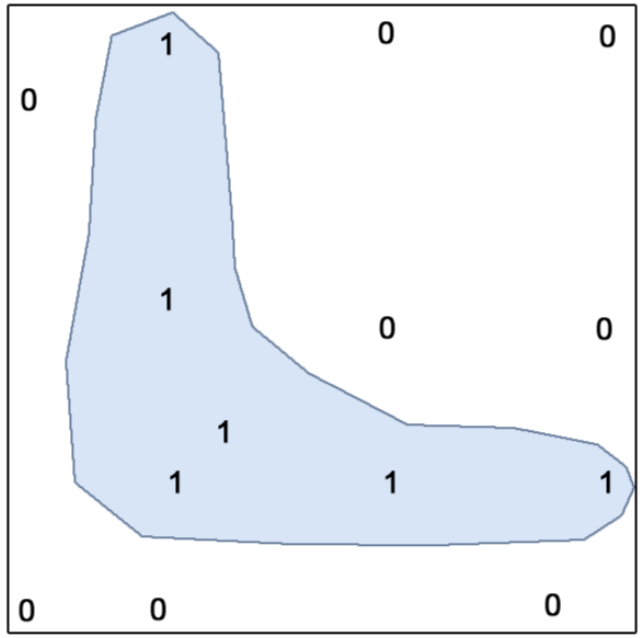
nicht austrainiert

austrainiert

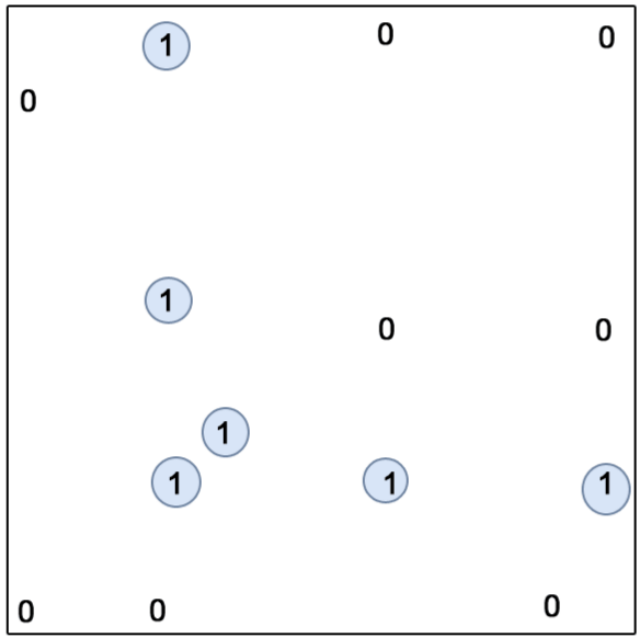
vermutlich kein gutes Modell?

Ziel ist eine gute Generalisierung

minimaler Fehler
und
gutes Trennpolygon



Overfitting entsteht, wenn das Modell zu stark an die Trainingsdaten angepasst wird



Ursachen für Überanpassung:

- zu komplexes neuronales Netz
- zu langes Training mit immer denselben Daten
- schlechte Trainingsdaten

minimaler Fehler
aber
keine Generalisierung

Kreuzvalidierung erlaubt die Mehrfachverwertung der Trainingsdaten



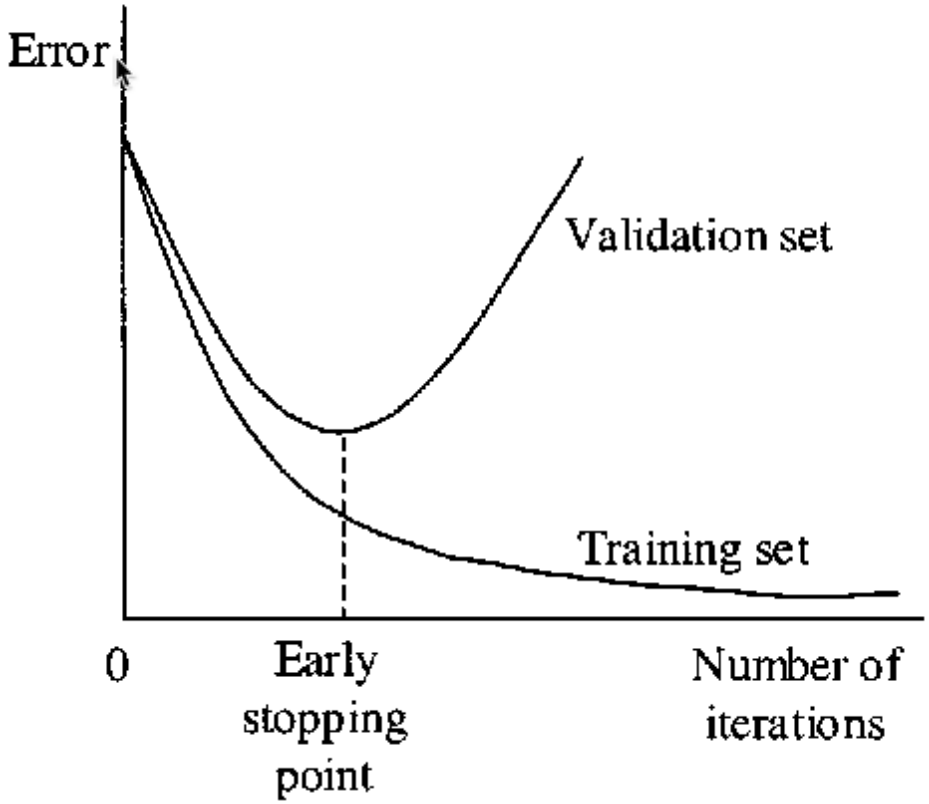
Außerdem möglich:

- Datenmenge vergrößern (rotieren, vergrößern, …)
- Neuronales Netz vereinfachen (dropout, regularization)

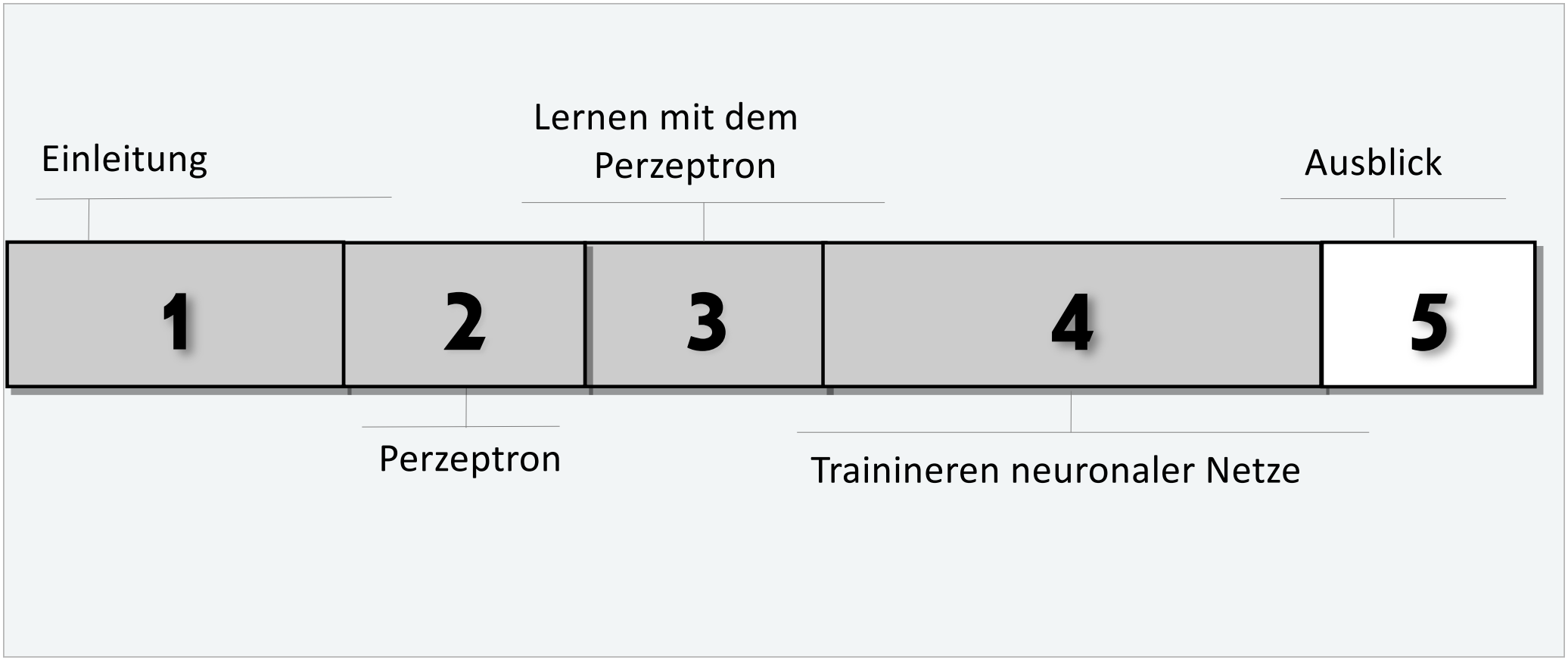
https://towardsdatascience.com/8-simple-techniques-to-prevent-overfitting-4d443da2ef7d

Beim early stopping wird der Fehleranstieg als Indikator für ein austrainiertes Netz verwendet

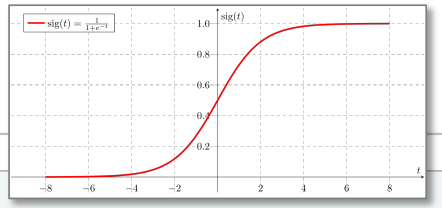
1. Aufteilung in Trainings- und Testdaten
2. große Zahl von Neuronen in verdeckter Schicht ...
3. ... mit kleinen Werten initialisieren
4. kleine Lernrate verwenden
5. Fehlerrate periodisch anhand der Testdaten bestimmen
6. stoppen, sobald die Fehlerrate ansteigt



Agenda maschinelles Lernen, Teil 1

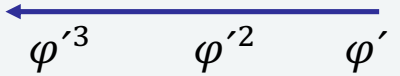


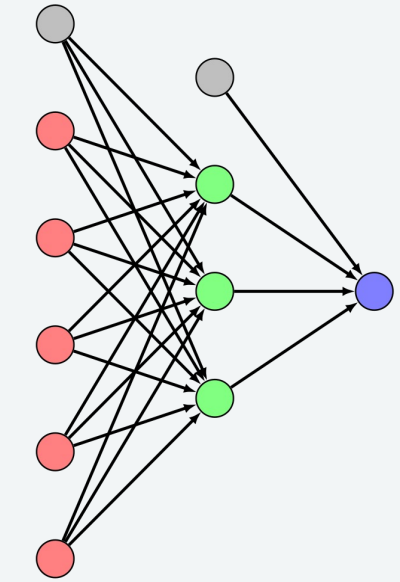
Problem des verschwindenden Gradienten in tiefen neuronalen Netzen



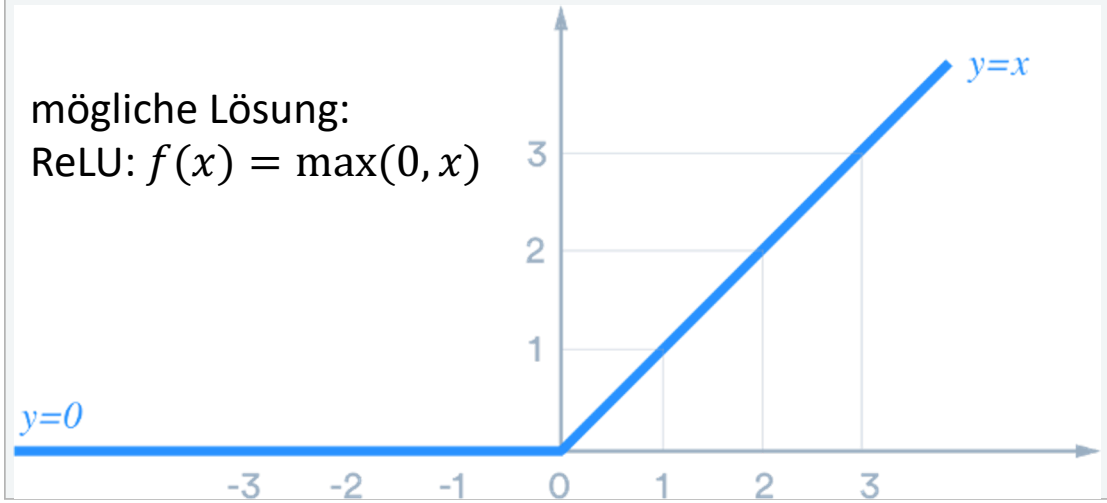
Abnehmende Gewichtsanpassung in Richtung Inputneuronen bei Sigmoid-Aktivierungsfunktion

$$\Delta w_{ij} = \alpha \cdot x_i \cdot \delta_j \text{ mit } \delta_j = \varphi'(net_j) \cdot \sum_k \delta_k \cdot w_{jk}$$





mögliche Lösung:
ReLU: $f(x) = \max(0, x)$



MartinThoma - Eigenes Werk, CC0, https://commons.wikimedia.org/w/index.php?curid=32970804000

Das Trainieren eines neuronalen Netzes ist ein empirischer Prozess

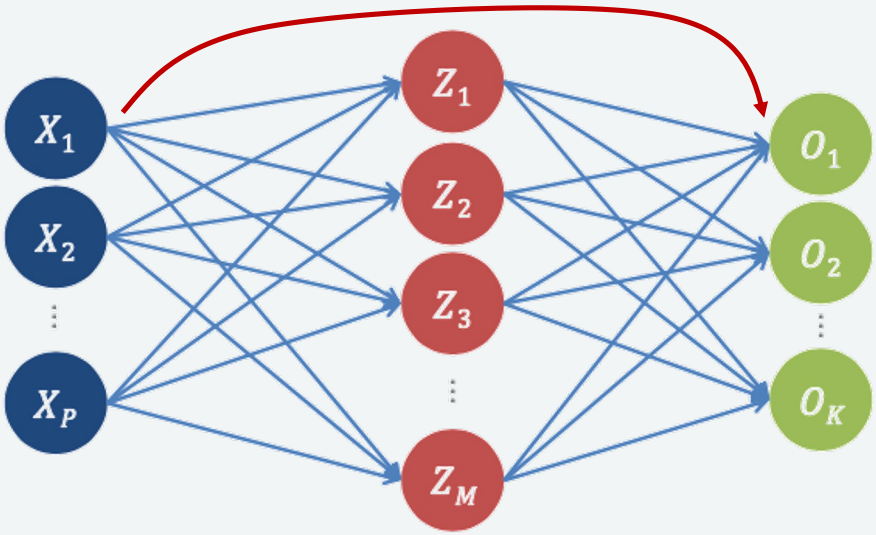
Trainingserfolg abhängig von

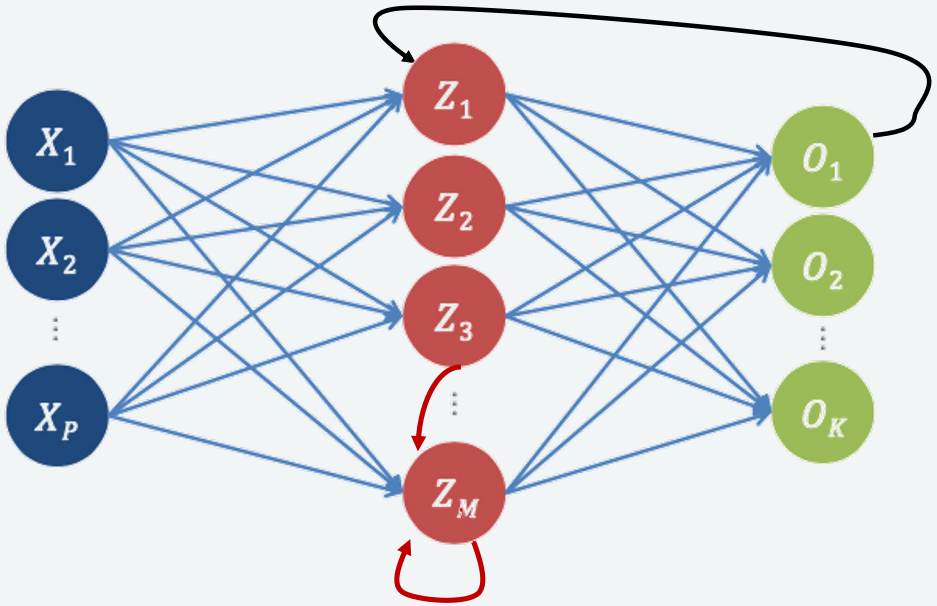
- Topologie, Aktivierungsfunktion,
- initialen Gewichten,
- Trainingsdaten,
- Trainingsprozess,
- Trainingsdauer,
- Lernrate.

} Trial and Error, bzw. Hyperparameteroptimierung

Andere Topologien für neuronale Netze

Feedforward-Netz mit Shortcut





rekurrentes Netz mit direkter und indirekter Rückkopplung



Dr.-Ing. Stefan Fricke

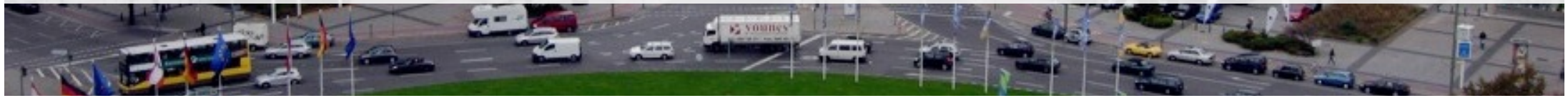
TU Berlin | Fachgebiet *Service-centric Networking*
Ernst-Reuter-Platz 1 | 10587 Berlin | Germany

stefan.fricke@tu-berlin.de

https://twitter.com/kuepp

https://www.xing.com/in/axelkuepper/

http://www.snet.tu-berlin.de/kuepper



Künstliche Intelligenz:
Grundlagen und Anwendungen

A|O|T
Agententechnologien in
betrieblichen Anwendungen
und der Telekommunikation

Maschinelles Lernen mit neuronalen Netzen
- Einführung -

Dr.-Ing. Stefan Fricke

Literatur und Links

neuromant.de/2018/12/16/Tutorial_Das-Perzeptron-Teil-3/

https://colah.github.io/posts/2015-08-Backprop/ (Mathematik hinter BP erklärt)

http://neuralnetworksanddeeplearning.com/chap1.html

David Kriesel: Ein kleiner Überblick über Neuronale Netze. 2007. Online: www.dkriesel.com/science/neural_networks, Lehrbuch in Deutsch und Englisch

Chi-Feng Wang: The Vanishing Gradient Problem - The Problem, Its Causes, Its Significance, and Its Solutions. 2019. Online: https://towardsdatascience.com/the-vanishing-gradient-problem-69bf08b15484

https://konfuzio.com/de/backpropagation/